



Lambda, Map, Filter & Reduce functions

By PrudviKrishna





Lambda Functions

Anonymous Functions:

Functions without a name, defined using the lambda keyword. Often used for short, simple operations.

Syntax:

1. lambda arguments: expression
2. lambda arguments: expression_if_true if condition else expression_if_false

Example1:

```
add = lambda x, y: x + y  
print(add(5, 3)) # Output: 8
```

Example2: usage of if else

```
check_even_odd = lambda x: "Even" if x % 2 == 0 else "Odd"  
print(check_even_odd(4)) # Output: Even
```



Map Function

The `map()` function applies a given function to each item in an iterable (like a list or tuple) and returns a map object (which is an iterator).

Syntax: `map(function, iterable)`

Example 1 : using Function

```
numbers = [1, 2, 3, 4]
```

```
def square(x):
```

```
    return x * x
```

```
result = map(square, numbers) # Applies square  
to each element
```

```
print(list(result)) # Output: [1, 4, 9, 16]
```

Example 2 : using lambda

```
numbers = [1, 2, 3, 4]
```

```
result = map(lambda x: x + 1, numbers)
```

```
for i in result:
```

```
    print(i)
```

```
print(list(result))
```



Filter function

The `filter()` function filters elements from an iterable based on a condition, returning only elements for which the condition is True.

Syntax:

```
filter(function, iterable)
```

Example: using function

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
def is_even(x):  
    return x % 2 == 0
```

```
result = filter(is_even, numbers)  
print(list(result)) # Output: [2, 4, 6]
```

Example 2 : using lambda

```
result = filter(lambda x: x%2==0,  
numbers)print(list(result)) # Output: [2, 4, 6]
```

Reduce Function

reduce() Function (from functools module)

The reduce() function is used to apply a function of two arguments cumulatively to the items of an iterable, from left to right, reducing the iterable to a single value.

Syntax:

```
from functools import reduce
reduce(function, iterable)
```

Example 1 :

```
from functools import reduce
numbers = [1, 2, 3, 4]
def multiply(x, y):
    return x * y
```

```
result = reduce(multiply, numbers)
print(result) # Output: 24 (1 * 2 * 3 * 4)
```

Example 2 : using lambda

```
result = reduce(lambda x, y: x + y, numbers)
print(result) # Output: 10 (1 + 2 + 3 + 4)
```

A large, abstract graphic consisting of several concentric, overlapping rings in shades of blue, green, and purple, framing the central text.

Thank you